

A digital signal processor-efficient accelerator for depthwise separable convolution

Xueming Li,¹ Hongmin Huang,¹ Yuan Liu,²

Xianghong Hu,² and Xiaoming Xiong²

¹School of Automation, Guangdong University of Technology, Guangzhou, China

²School of Microelectronics, Guangdong University of Technology, Guangzhou, China

Email: xianghonghu@gdut.edu.cn; xmxiong@gdut.edu.cn

Recent researches on deep convolution neural networks have proposed some compact networks, such as MobileNet, but its main computation, depthwise separable convolution (DWC), which reduces the reusable data and improves the requirement of data loading efficiency. Although DWC can effectively reduce the amount of network computation, it needs a special accelerator to enhance the inference speed. This paper proposes a high-performance accelerator for DWC based on the commonly used acceleration platform field-programmable gate array. The proposed accelerator supports the computation of both standard convolutions (SCs) and DWC as well as two activation functions. In addition, two data storage formats are used to maintain the data loading efficiency for different input requirements of SC and DWC under high parallelism. Furthermore, a processing unit that can execute two 8×8 -bit multiplications inside one digital signal processor (DSP) is designed to make the best use of the DSP hardware resources. Finally, the accelerator is implemented on ZYNQ ZC706 at 200 MHz. Consuming only 392 DSPs, the accelerator achieves 134.5 giga operations per second (GOPS) and 209.4 frames per second (FPS) on MobileNet V1 as well as 96.4 GOPS and 250.4 FPS on MobileNet V2. Experimental results demonstrate that this design provides a better DSP efficiency than previous works.

Introduction: The deep convolution neural network (DCNN) achieves performance that traditional algorithms cannot match on many tasks, such as speech recognition [1], object detection [2], semantic segmentation [3], and so forth. However, these networks are computationally expensive (over billions of multiply-and-accumulate) and have a large number of parameters (more than 10 million parameters), making them difficult to implement on embedded platforms. Therefore, some studies try to design a compact network, such as MobileNet [4], which requires only 4.2 million parameters and achieves the accuracy of 70.6% on the ImageNet-2012 dataset.

The main calculation of MobileNet is depthwise separable convolution (DWC), which can greatly reduce the amount of calculation and parameters compared with the standard convolution (SC). However, the reduction of computational density increases the requirement of memory access bandwidth since there are no reusable data across different output channels in depthwise convolution, which makes DWC difficult to effectively utilize the hardware resources. Therefore, it is necessary to design an accelerator that can maintain high performance in both DWC and SC.

Field programmable gate array (FPGA) provides abundant hardware resources and can quickly implement complex circuit design. Also, FPGA can quickly update the design with its programmability and can be deployed quickly. Therefore, there are several researches on DCNN accelerator based on FPGA [9, 10].

In this paper, we present a hardware-efficient DCNN accelerator based on Xilinx FPGA that can accelerate both SC and DWC with high hardware resources utilization. In addition, two data storage formats are adopted to improve the efficiency of data loading of two types of convolutions. Besides, a DSP-efficient processing unit (PU) is proposed to maximize the utilization of DSP resources.

Deep convolution neural network: DCNN mainly consists of convolution layer, pooling layer, batch normalization layer and fully connected layers. However, the convolution layer accounts for over 90% of total network calculation [5]. Therefore, the proposed accelerator mainly focuses on accelerating the convolution layers. Moreover, the pooling layer

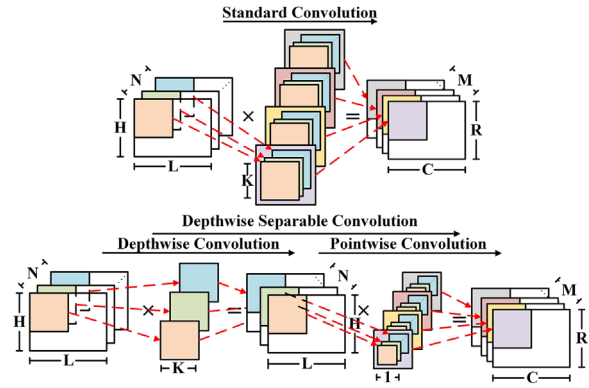


Fig. 1 Computation of standard convolution and depthwise separable convolution

can be replaced by a convolution layer with a stride of 2, and the fully connected layer can be considered as a special 1×1 convolution layer, and the batch normalization layer can be fused into the previous convolution layer.

It is proposed in [11] that replacing large convolution kernel sizes like 7×7 and 5×5 with multiple 3×3 convolutions cascaded can obtain the same receptive field and reduce the amount of computation and parameters of the network. Thus, the accelerator designed in this paper only supports 3×3 and 1×1 SC and DWC with the stride of 1 or 2.

The computation of DWC and SC is shown in Figure 1. DWC decomposes the SC into depthwise convolution and pointwise convolution. Depthwise convolution is a 2D convolution and the pointwise convolution is just a 1×1 SC. When the kernel size $K = 3$ and input and output channel $N = M = 256$, it can reduce 88.5% of both calculation and parameters.

The convolution is usually followed by an activation function that provides nonlinearity. The most common activation functions are Rectified Linear Units (ReLU) and Leaky ReLU, as shown in Formulas (1) and (2), where the typical value of α is 0.1. However, it is complex to multiply by 0.1 in hardware, so we set $\alpha = 0.125 = 2^{-3}$, and multiplying by 0.125 in hardware can be achieved by shifting right by three bits:

$$\text{ReLU} = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases} \quad (1)$$

$$\text{Leaky ReLU} = \begin{cases} x, & x < 0 \\ \alpha x, & x \geq 0 \end{cases} \quad (2)$$

Accelerator architecture: The architecture of the accelerator proposed in this paper is shown in Figure 2. The whole acceleration process is as follows: the CPU transmits the input feature maps (IFMs) to the feature map buffer 1 (FM BUF-1) through the AXI bus and configures the CONTROL module according to the running network. Then the COMPUTING CORE reads the images and weights from the corresponding buffers to the register array. Next, starts the PU ARRAY for convolution calculation. The final convolution results are stored to another FM BUF for the computation of the next layer or stored to the RESULT FIFO to return the results to the CPU.

Data storage format and feature map buffer: A 3D image has two commonly used storage formats, channel first (CF) and channel last (CL). The CF format stores the data of the same CF, that is, the same row of data is stored in adjacent addresses, which is suitable for SC that has a large amount of reusable data. The CL format stores the data of different channels first, with the data of adjacent channels at the same position being stored in the adjacent addresses, which is suitable for the depthwise convolution with less reusable data.

In SC, the IFMs can be reused across different output channels, and the weights can be reused inside the same output channels, which means that $7 \times 7 \times 8$ PU ARRAY only needs $7 \times 7 \times 1$ IFMs and $1 \times 1 \times 8$ weights for each calculation. Using CF format, the same row IFM data can be read efficiently by increasing the bitwidth of the read port. In DWC, only weights can be reused, which means that $7 \times 7 \times 8$ IFMs

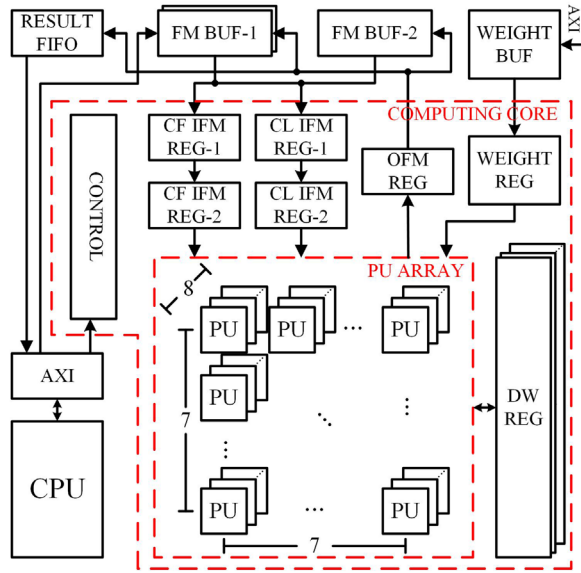


Fig. 2 Architecture of proposed accelerator

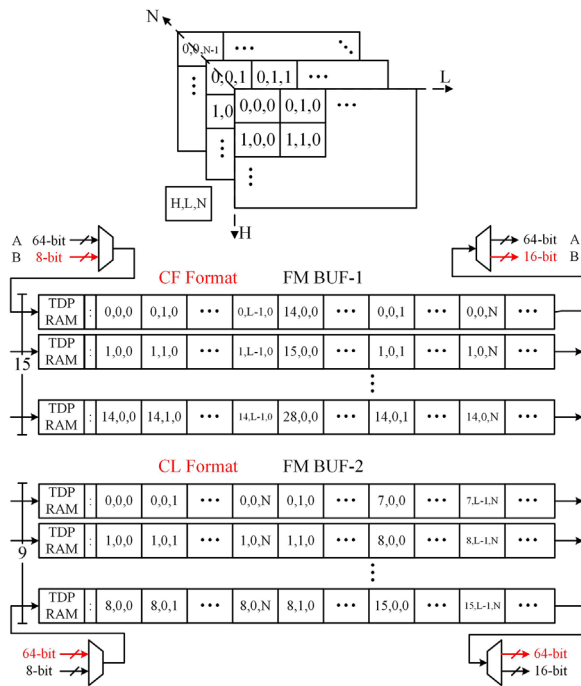


Fig. 3 Two-storage format of feature map buffer

are required for each calculation. Using CL format, the data of adjacent output channels can be quickly read, and the input requirements of the PU ARRAY can be met by increasing the bitwidth of the read port even if there is no reusable data among different output channels.

The FM BUF-1 is composed of 15 True Dual-Port RAMs (TDP RAMs), and the FM BUF-2 is composed of nine TDP RAMs, each ram has two reads and write ports, corresponding to two data storage formats, as shown in Figure 3. All the data is quantized into an 8-bit fixed-point. The read port bitwidth of CF format (marked as port A) is 16-bit, and the write port bitwidth is 8-bit. While the bitwidth of the read port and write port of CL format (marked as port B) are both 64-bit, eight channels of IFM can be read in one clock.

When computing 3×3 SC with the stride of 2, the PU ARRAY can calculate $7 \times 7 \times 16$ (row $\times$ column $\times$ channel) OFMs in parallel, two channels per PU, which needs 15×15 IFM data for each input channel. It can be read from buffers in seven or eight clocks by port A, which is less than the convolution time of nine clocks. When computing 3×3 DWC with the stride of 1, the PU ARRAY calculates $7 \times 7 \times 8$ output pixels in parallel that needs $9 \times 9 \times 8$ IFM data and no data can be reused across different input channels. nine clocks are needed to read

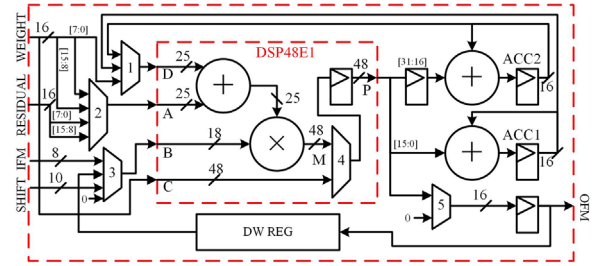


Fig. 4 Architecture of processing unit

Table 1. Data selection in different operations

Multiplexer	1	2	3	4	5
SC	WEIGHT	WEIGHT	IFM	M	0
DWC	0	WEIGHT	DW REG	M	0
Bias	0	0	0	C	0
ReLU-1	ACC1	0	SHIFT	M	P/0
ReLU-2	ACC2	0	SHIFT	M	P/0
Leaky ReLU-1	ACC1	0	SHIFT	M	P/0
Leaky ReLU-2	ACC2	0	SHIFT	M	P/0
Residual-1	0	RESIDUAL	SHIFT	M	0
Residual-2	0	RESIDUAL	SHIFT	M	0

Abbreviations: SC, standard convolution; DWC, depthwise separable convolution; IFM, input feature map; ReLU, Rectified Linear Units.

IFM from buffers by port B, just equal to the convolution time, which means the data load time can also be covered.

Computing core and processing unit: As is shown in Figure 2, the COMPUTING CORE consists of a CONTROL module, a set of register arrays and a PU ARRAY. The CONTROL module controls the PU ARRAY to compute the convolution according to the convolution parameters and generates the data addresses of the FM BUF and WEIGHT BUF for data loading. The register arrays provide the required data for the convolution of PU ARRAY.

The PU ARRAY consists of $7 \times 7 \times 8$ PUs, and each PU uses only one DSP, as shown in Figure 4 where the selections of multiplexers in different operations are listed in Table 1. When computing SC and pointwise convolution, the 18×25 -bit multiplier inside DSP can calculate two 8×8 -bit output pixels in parallel. The 16-bit input WEIGHT consists of two weights of the adjacent output channel, WEIGHT[15: 8] and WEIGHT[7: 0], which need to be separated to avoid overlap of the multiplication results using multiplexer 1 and 2. The two weights are then added by the adder inside DSP, separated by 8-bit. While computing depthwise convolution, only WEIGHT[7: 0] is valid, and the output of the PU ARRAY is temporarily stored in the DW register for the preparation of the upcoming pointwise convolution. The DW register can provide IFM data to each PU every clock which can greatly improve the computational efficiency of pointwise convolution.

The fixed-point output usually requires adjusting the number of bits that represent the decimal. And the Leaky ReLU also needs shift operation. To simplify the data selection, the adjustment and the shift operation in the activation function are computed by the multiplier. The output is shifted 8-bit to the right in ReLU and 11-bit in Leaky ReLU. And the parameter SHIFT is a power of 2, which will multiply by the output pixel to control the number of bits shifted to the right. The number of bits shifted to the right is $8 - \log_2 \text{SHIFT}$ or $11 - \log_2 \text{SHIFT}$. Therefore, the barrel shifter which is an efficient way to shift data can be reduced into a data multiplexer.

Experimental section: The proposed accelerator is designed based on Verilog HDL and implemented on the Xilinx ZYNQ-7 ZC706 evaluation board using Vivado 2018.3. The implementation result shows that the accelerator consumes 31.27% LUTs, 21.40% FFs, 453.5 BRAMs and 392 DSPs at a working frequency of 200 MHz.

MobileNet V1 [4] is the first DCNN to use DWC and MobileNet V2 [12] is its improved version. These two networks contain both SC and

Table 2. Statistics of MobileNet

	MobileNet V1	MobileNet V2
#SC layers	15	36
#DWC layers	13	17

Abbreviations: SC, standard convolution; DWC, depthwise separable convolution.

Table 3. Performance under different width multiplier and resolution multiplier

Width multiplier	Resolution multiplier	Input size	GOPS	FPS
1	1	224 × 224	134.5	209.4
0.75	1	224 × 224	129.7	337.5
0.5	1	224 × 224	124.4	639.8
1	0.86	192 × 192	104.9	220.8
1	0.71	160 × 160	81.8	246.0

Table 4. Comparisons of performance

MobileNet V1				
		[8]	This work	
Device		XCK325T		XC7Z020
Precision		8-bit		8-bit
Frequency (MHz)		200		200
Perf. (GOPS)		147.8		134.5
DSPs		704		392
Perf. / DSPs		0.21		0.343
FPS		264.6		209.4
MobileNet V2				
	[6]	[7]	[8]	This Work
Device	Arria 10 Soc	Stratix V SGSD8	XCK325T	XC7Z020
Precision	16-bit	16-bit	8-bit	8-bit
Frequency (MHz)	133	160	200	200
Perf. (GOPS)	170.6	73.6	98.6	96.4
DSPs	1278	576	704	392
Perf. / DSPs	0.2 ^a	0.165 ^a	0.14	0.246
FPS	170.6	-	325.7	250.4

^aNormalized to 200 MHz.

DWC, the detailed statistic is listed in Table 2. Although MobileNet V2 has more layers, its calculation is only 55% of MobileNet V1 and it has more DWC layers.

To improve the universality of the network, MobileNet introduced two hyperparameters, width multiplier to control the number of feature map channels and resolution multiplier to control the size of the input image. The performance under different width multipliers and resolution multipliers are shown in Table 3.

Because the minimum channel parallelism of the PU ARRAY is 16, the PU ARRAY can always maintain a full load state as long as the number of output channels is a multiple of 16. As the layers deepen, the number of channels is usually large, so the width multiplier has no significant impact on the hardware performance. However, the reduction of resolution will make the minimum size of the feature map less than 7×7 , resulting in a waste of hardware resources and performance degradation.

The comparison with prior works is listed in Table 4. For a fairer comparison, the performance per DSP is normalized to 200 MHz. The proposed design has the highest DSP efficiency of 0.343 GOPS/DSP on MobileNet V1. The CL format and DW register have a significant impact on improving the calculation efficiency of DWC. Also, the DSP-efficient PU is key to high performance. Although the designs of [6] and [8] have higher performance on MobileNet V2 because of using more DSPs, their DSP efficiencies are lower than our design.

Furthermore, our design can achieve a higher FPS if using a similar number of DSPs in [8]. One of the most direct ways is to use more DSPs

to increase the channel parallelism of the PU ARRAY from $7 \times 7 \times 8$ to $7 \times 7 \times 16$. But it makes the accelerator more sensitive to the width multiplier. Another way is to implement two or more separate COMPUTING CORE, which requires additional on-chip storage resources but enables parallelism between adjacent frames. Thus, our design also has scalability.

Conclusion: This paper proposes a DSP-effective accelerator for DWC and SC with high performance. Two data storage formats are introduced to improve the data loading efficiency of both DWC and SC, and a DW register is used to further improve the performance of wise convolution. DSP efficiency is highly improved by computing two 8×8 -bit multiplications simultaneously, which makes the best use of the 18×25 -bit multiplier inside DSP48E1 under 8-bit fixed-point quantization. The experiments show that our design achieves a higher DSP efficiency compared with the prior works.

There are still works that can continue to improve this design. The hardware needs to maintain high performance under different resolution multipliers. Moreover, the efficient data transmission scheme and data tiling scheme can reduce the gap between the theoretical performance and the actual performance of the hardware.

Acknowledgement: This work was supported by the Key-Area Research & Development Program of Guangdong Province, China under Grant 2021B0101410004.

Conflict of interest: The authors declare that there is no conflict of interests, we do not have any possible conflicts of interest.

© 2022 The Authors. *Electronics Letters* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology

This is an open access article under the terms of the Creative Commons Attribution-NonCommercial-NoDerivs License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

Received: 26 October 2021 Accepted: 17 January 2022

doi: 10.1049/ell2.12435

References

- Swietojanski, P., Ghoshal, A., Renals, S.: Convolutional neural networks for distant speech recognition. *IEEE Signal Process. Lett.* **21**(9), 1120–1124 (2014)
- Zhu, H., Yan, X., Tang, H., Chang, Y., Li, B., Yuan, X.: Moving object detection with deep CNNs. *IEEE Access*. **8**, 29729–29741 (2020)
- Hu, X., Wang, H.: Efficient fast semantic segmentation using continuous shuffle dilated convolutions. *IEEE Access*. **8**, 70913–70924 (2020)
- Howard, A.G. et al.: MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *Computer Vision and Pattern Recognition* (2017).
- Cong, J., Xiao, B.: *Minimizing Computation in Convolutional Neural Networks*. Springer, Cham (2014)
- Bai, L., Zhao, Y., Huang, X.: A CNN accelerator on FPGA using depthwise separable convolution. *IEEE Trans. Circuits Syst. II Express Briefs* **65**(10), 1415–1419 (2018)
- Xie, X. et al.: Fast-ABC: a fast architecture for bottleneck-like based convolutional neural networks. *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2019
- Yunxuan, Y. et al.: Light-OPU: An FPGA-based overlay processor for lightweight convolutional neural networks. *The ACM/SIGDA International Symposium on Field-Programmable Gate Arrays ACM*, 2020
- Pestana, D. et al.: A full featured configurable accelerator for object detection with YOLO. *IEEE Access* **9**, 75864–75877 (2021)
- Huang, W. et al.: FPGA-based high-throughput CNN hardware accelerator with high computing resource utilization ratio. *IEEE Trans. Neural Networks Learn. Syst.* 1–15 (2015)
- Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. *Computer Vision and Pattern Recognition* (2015)
- Sandler, M. et al.: MobileNetV2: Inverted residuals and linear bottlenecks. *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2018)